

Git Workflow Guide

PSID Project

Sarah Sullivan

Created: March 25, 2026

Last Updated: March 25, 2026

Made with: GitHub Copilot / Claude Sonnet 4.6

This file summarizes the command line statements to push a commit to GitHub.

VS Code setting: `settings.json` is configured with `latex-workshop.latex.autoClean.run: "onBuilt"` — auxiliary `.tex` files (`.aux`, `.log`, `.toc`, etc.) are automatically deleted after every compilation.

Contents

I. Workflow	2
1) Everyday: push VS Code to GitHub	2
2) Collaborating: push GitHub changes to VS Code	2
II. Set Up	2
1) Login	2
2) Navigate to repo folder	2
3) Initialize	2
4) Edit <code>.gitignore</code>	2
5) Stage and commit	3
6) Connect to remote (first time only)	3
7) Push to GitHub	3
III. Troubleshooting	3
1) If push is rejected (histories don't match)	3
2) Lock file error	4
3) Unstage a file	4
4) Check status / history / remotes	4
5) See what changed	4
6) After updating <code>.gitignore</code> : untrack already-tracked files	4
7) If push is rejected because GitHub has newer changes (stash workflow)	4
8) Open files & folders	5
9) Navigate directories	5
10) Compile LaTeX	5
11) File operations	5

I. Workflow

1) Everyday: push VS Code to GitHub

```
git pull                # get latest changes
git add .               # stage changes
git commit -m "comment changes"
git push                # upload to GitHub
```

2) Collaborating: push GitHub changes to VS Code

```
git pull origin main
git pull
```

II. Set Up

1) Login

```
git config --global user.name "Your Name"
git config --global user.email "sbsullivan19@gmail.com"
```

2) Navigate to repo folder

```
cd "/Users/sarsul/Library/CloudStorage/Dropbox-
UniversityofMichigan/Sarah Sullivan/SARAH-SOFT/research/psid"
```

3) Initialize

```
git init
git branch -M main
```

4) Edit .gitignore

Open .gitignore in VS Code:

```
code .gitignore
```

Add files and folders to exclude from the repository. Currently, I am only uploading .do files to the repository.

After editing, stage and commit .gitignore:

```
git add .gitignore
git commit -m "update gitignore"
```

If files were **already tracked** by Git before you added them to `.gitignore`, Git will keep tracking them. Untrack a specific file without deleting it:

```
git rm --cached filename.dta
```

Untrack everything and re-add (use carefully):

```
git rm --cached -r .
git add .
git commit -m "remove tracked files now in gitignore"
```

5) Stage and commit

```
git add .
git status          # check what will be committed
git commit -m "COMMIT MESSAGE"
```

6) Connect to remote (first time only)

```
git remote add origin https://github.com/sarahs-house/psid.git
```

Check it worked:

```
git remote -v
```

7) Push to GitHub

```
git push -u origin main
```

After the first push, you can just use:

```
git push
```

III. Troubleshooting

1) If push is rejected (histories don't match)

```
git fetch origin
git rebase origin/main
```

Or if you need to force-merge unrelated histories:

```
git pull origin main --allow-unrelated-histories
git push -u origin main
```

2) Lock file error

```
fatal: Unable to create '.git/index.lock': File exists.
```

Fix:

```
rm -f .git/index.lock
```

3) Unstage a file

```
git restore --staged filename.do
```

4) Check status / history / remotes

```
git status
git log --oneline
git remote -v
```

5) See what changed

```
git diff
```

6) After updating .gitignore: untrack already-tracked files

If you add a file type to `.gitignore` but Git was already tracking those files (i.e., they were previously committed), Git will keep tracking them. You need to explicitly untrack them.

First, see what is currently tracked that you want to remove:

```
git ls-files | grep -E "\.(aux|log|toc|out|fls|synctex\.gz|fdb_latexmk)$"
```

Untrack specific files (they stay on your computer, just removed from GitHub):

```
git rm --cached file1.aux file2.log    # list each file to untrack
```

Then commit and push the removal:

```
git add .gitignore
git commit -m "remove tracked files now in gitignore"
git push
```

7) If push is rejected because GitHub has newer changes (stash workflow)

This happens when GitHub has commits you don't have locally — e.g., you pushed from another machine, or edited a file on GitHub directly. Your local commit exists but Git refuses to push because the histories have diverged.

Step 1: Stash any uncommitted local changes so they don't block the pull:

```
git stash
```

Step 2: Pull and rebase (replays your local commit on top of the remote changes):

```
git pull --rebase origin main
```

Step 3: Restore your stashed changes:

```
git stash pop
```

Step 4: Push:

```
git push
```

Or as one line:

```
git stash && git pull --rebase origin main && git stash pop && git push
```

8) Open files & folders

```
code .                # open current folder in VS Code
code filename.do       # open a specific file in VS Code
open filename.pdf      # open file in default app (e.g.
    PDF viewer)
open -a "TeXShop" file.tex  # open file in a specific app
```

9) Navigate directories

```
pwd                    # print current directory
ls                     # list files in current directory
ls -la                # list files with details and
    hidden files
cd foldername          # go into a folder
cd ..                  # go up one level
cd -                   # go back to previous directory
```

10) Compile LaTeX

```
pdflatex filename.tex  # compile .tex to PDF (run twice
    for TOC)
```

11) File operations

```
mv oldname.do newname.do  # rename a file
cp file.do copy_of_file.do # copy a file
```

```
rm filename                # delete a file (careful -- no  
                           undo!)  
mkdir foldername           # create a new folder
```